



Security Review For Townsquare



Collaborative Audit Prepared For:
Lead Security Expert(s):

Townsquare
0xDjango
Inc0gn170

Date Audited:

January 20 - January 22, 2026

Introduction

TokenMinter is the protocol's emissions and distribution controller for the Town token. It deploys a new Town token instance, initializes fixed allocation "buckets" (Investor, Team, Foundation, Community, Ecosystem, Liquidity) as basis-point shares of the token's initial total supply, and enforces a team-controlled administration model for setting emission parameters and distributing tokens from each allocation. On each epoch transition, `updatePeriod()` computes a growth amount derived from voting-escrow supply data, transfers the growth portion to the rewards distributor for rebasing/accounting, and notifies the voter contract of the epoch's emissions for downstream gauge/reward distribution.

Scope

Repository: `TowneSquare/TokenMinter`

Audited Commit: `63700a1ddf0d8fa09eb86db9f2f1e15aeec94fff`

Final Commit: `2efb6112d57267f22ac699b19d5f9397f9cec806`

Files:

- `foundry.lock`
- `foundry.toml`
- `.github/workflows/test.yml`
- `.gitignore`
- `.gitmodules`
- `README.md`
- `remappings.txt`
- `script/TokenMinter.s.sol`
- `src/governance/IVotes.sol`
- `src/interfaces/IRewardsDistributor.sol`
- `src/interfaces/ITokenMinter.sol`
- `src/interfaces/ITown.sol`
- `src/interfaces/IVoter.sol`
- `src/interfaces/IVotingEscrow.sol`
- `src/TokenMinter.sol`
- `src/Town.sol`
- `test/TokenMinter.t.sol`

Final Commit Hash

2efb6112d57267f22ac699b19d5f9397f9cec806

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	1	3

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue M-1: activePeriod set incorrectly due to switched mathematical operations [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-01-townsquare-jan-20th/issues/14>

Summary

The calculation of `activePeriod` is incorrect when calling `setEpochDuration()` because the function multiplies by `epochDurationInSeconds` prior to dividing by `epochDurationInSeconds`. The correct order is to divide and then multiply to take advantage of the precision loss to anchor the `activePeriod` variable to the start of the period.

Vulnerability Detail

When initially setting the epoch duration via `setEpochDuration()`, the `activePeriod` is supposed to be set to the first timestamp of the current period. For example, if the period is 1 week, calling `setEpochDuration()` should set the `activePeriod` to the exact second the week began.

The correct implementation is:

```
activePeriod = (block.timestamp / epochDurationInSeconds) * epochDurationInSeconds;
```

A week contains 604800 seconds. Imagine the current timestamp is 1769045376.

$(1769045376 / 604800) * 604800 = 1769040000$ which represents the start of the period.

However, the calculation is currently implemented as:

```
activePeriod = ((block.timestamp) * epochDurationInSeconds) / epochDurationInSeconds;
```

This firstly multiplies by the `epochDuration` and then divides by the same value.

$(1769045376 * 604800) / 604800 = 1769045376$

Impact

The first `activePeriod` will not be pegged to the start of the period.

Code Snippet

<https://github.com/sherlock-audit/2026-01-townsquare-jan-20th/blob/63de742027c55b1e9e7e62e86f7231f23c813939/TokenMinter/src/TokenMinter.sol#L127C9-L127C94>

Tool Used

Manual Review

Recommendation

Correct the calculation to `activePeriod = ((block.timestamp) * epochDurationInSeconds) / epochDurationInSeconds;`

Issue L-1: Unreachable functions due to token supply cap [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-01-townsquare-jan-20th/issues/15>

Summary

The Town.sol ERC20 contract contains a `mint()` function that allows a minter to mint more tokens to an arbitrary address. However, since the totalSupply is capped at 10_000_000_000 ether and this amount is minted in the constructor, any attempt to call the `mint()` function will revert with a `ERC20ExceededCap` error.

Vulnerability Detail

The Town.sol contract inherits from `ERC20Capped.sol` which sets a maximum total supply that can be minted. Since the entire total supply is minted in the constructor, the `mint()` function is unusable.

The function `setMinter()` is also useless because minters are not able to call the `mint()` function.

Impact

Unreachable code.

Code Snippet

<https://github.com/sherlock-audit/2026-01-townsquare-jan-20th/blob/63de742027c55b1e9e7e62e86f7231f23c813939/TokenMinter/src/Town.sol#L22-L31>

Tool Used

Manual Review

Recommendation

Remove the `mint()` and `setMinter()` functions.

Issue L-2: Missing Events on Critical State Changes [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-01-townsquare-jan-20th/issues/16>

Summary

Critical admin functions `setVoter()`, `setVe()`, `setRewardsDistributor()`, and `setEpochDuration()` do not emit events when modifying protocol state.

Vulnerability Detail

These functions modify core protocol addresses and parameters but provide no on-chain record of changes:

```
function setVoter(address _voter) external onlyTeam {
    voter = IVoter(_voter);
    // No event emitted
}

function setVe(address _ve) external onlyTeam {
    ve = IVotingEscrow(_ve);
    // No event emitted
}

function setRewardsDistributor(address _rewardsDistributor) external onlyTeam {
    rewardsDistributor = IRewardsDistributor(_rewardsDistributor);
    // No event emitted
}

function setEpochDuration(uint256 _epochDuration) external onlyTeam {
    epochDurationInSeconds = _epochDuration;
    activePeriod = ((block.timestamp) * epochDurationInSeconds) /
        ↪ epochDurationInSeconds;
    // No event emitted
}
```

Impact

- Off-chain monitoring cannot detect critical parameter changes
- No audit trail for administrative actions

Tool Used

Manual Review

Recommendation

Add events for each function.

Issue L-3: Missing Implementations: pause/unpause, finalizeDestinations, recoverERC20 [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-01-townsquare-jan-20th/issues/17>

Summary

The contract is missing three functions specified in the design document: `pause()`/`unpause()`, `finalizeDestinations()`, and `recoverERC20()`.

Vulnerability Detail

1. Missing `pause()`/`unpause()`: No emergency stop mechanism exists. Emissions cannot be halted during exploits or governance migrations.
2. Missing `finalizeDestinations()`: Destination addresses (`voter`, `ve`, `rewardsDistributor`) can be changed indefinitely. No mechanism to lock them after setup.
3. Missing `recoverERC20()`: Accidentally sent tokens are permanently locked. No recovery mechanism for non-TOWN tokens.

Tool Used

Manual Review

Recommendation

Consider implementing the above functions.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.